WEEK 2 · LESSON 9 OF 24 · HANDOUT

Monitoring Models in Production

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=n0bR0IArJDo — open the lesson online to watch inside the course.

🎓 **Go deeper:** EuroPython 2023: Stop using print! Understanding and using the logging module — https://www.youtube.com/watch?v=5f4o3nyAz5w

🎓 **Go deeper:** PyCon DE 2023: Staying Alert — How to Implement Continuous Testing for Machine Learning Models — https://www.youtube.com/watch?v=4BgmBtsS0iA

Your goal in this lesson: Log predictions safely, compute service and model health metrics over time, and set alert rules that catch problems quickly.

What to watch

Layer	Metrics	Example alert
Service	Requests, errors, latency (p50, p95)	Error rate above 2%
Inputs	Missing values, ranges, distribution (drift)	Income distribution moved (next lesson)
Outputs	Approval rate, average probability	Approval rate jumps from 70% to 40%
Quality	Accuracy when real outcomes arrive	Accuracy 5 points below launch
Fairness	Approval rate by group (Week 3)	Gap between groups grows

Outcomes (did they repay?) arrive **months** later, so service, input and output metrics are your early warning.

Log every prediction — safely

```
import hashlib
import hmac
import json

SECRET = b"replace-with-a-secret-from-the-environment"

def log_line(applicant_id, features, probability, latency_ms, status, version="1.0.0"):
    return json.dumps({
        "ts": "2026-11-09T10:15:02Z",
        "applicant": hmac.new(SECRET, str(applicant_id).encode(), hashlib.sha256).hexdigest()
[:16],
        "version": version, "status": status, "latency_ms": latency_ms,
        "probability": probability, "features": features,
    })

print(log_line(1042, {"income_rwf": 350000, "loan_rwf": 200000}, 0.91, 38, 200))
```

The applicant's ID is replaced by a keyed hash (Week 3 explains why): you can still link a log line to a later outcome, but someone reading the logs cannot see who it was. Never log names, phone numbers or national IDs. Structured **JSON lines** are easy to search and analyse.

Two weeks of traffic

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]
TARGET = "repaid"
train, test = train_test_split(applicants, test_size=0.25, random_state=0,
stratify=applicants[TARGET])
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(train[FEATURES], train[TARGET])
proba = model.predict_proba(test[FEATURES])[:, 1]
print(f"accuracy {accuracy_score(test[TARGET], proba >= 0.5):.3f} | AUC
{roc_auc_score(test[TARGET], proba):.3f}")
```

We simulate 14 days of requests. On day 10 the database becomes slow; on day 12 a new app version sends some broken requests:

```
rng = np.random.default_rng(0)
logs = []
for day in range(1, 15):
    batch = make_applicants(n=300, seed=1000 + day)
    p = model.predict_proba(batch[FEATURES])[:, 1]
    latency = rng.lognormal(np.log(40), 0.35, len(batch)) * (3.0 if day == 10 else 1.0)
    status = np.where(rng.random(len(batch)) < (0.08 if day == 12 else 0.005), 422, 200)
    logs.append(pd.DataFrame({"day": day, "latency_ms": latency.round(1), "status": status,
                              "probability": np.where(status == 200, p, np.nan)}))
logs = pd.concat(logs, ignore_index=True)

daily = logs.groupby("day").agg(
    requests=("status", "size"),
    error_rate=("status", lambda s: (s != 200).mean()),
    p50_ms=("latency_ms", "median"),
    p95_ms=("latency_ms", lambda s: s.quantile(0.95)),
    approve_rate=("probability", lambda s: (s.dropna() >= 0.5).mean()),
).round(3)
print(daily.to_string())
```

p95 latency means 95% of requests were faster than this. Averages hide slow requests; percentiles show what your slowest users feel.

Alert rules

```
RULES = {
    "error rate above 2%": lambda d: d["error_rate"] > 0.02,
    "p95 latency above 150 ms": lambda d: d["p95_ms"] > 150,
    "approval rate outside 60%-80%": lambda d: not 0.60 <= d["approve_rate"] <= 0.80,
}
for day, row in daily.iterrows():
    fired = [name for name, rule in RULES.items() if rule(row)]
    if fired:
        print(f"day {day}: ALERT -> {', '.join(fired)}")
```

```
import matplotlib.pyplot as plt
fig, axes = plt.subplots(1, 3, figsize=(13, 3.2))
for ax, col, limit in zip(axes, ["error_rate", "p95_ms", "approve_rate"], [0.02, 150, None]):
    ax.plot(daily.index, daily[col], marker="o"); ax.set_title(col); ax.set_xlabel("day")
    if limit: ax.axhline(limit, color="red", linestyle="--")
plt.tight_layout(); plt.show()
```

The alerts fire on exactly the bad days, and the approval-rate rule stays quiet because the model itself is fine. A good alert is **specific** (says what broke), **actionable** (someone knows what to do) and **rare** (people ignore alerts that fire every day).

In real systems

- Send logs to a central place (for example the ELK stack, Grafana Loki or a cloud service).
- Show dashboards with Grafana or similar, and send alerts to email, Slack or SMS.
- Write a **runbook** for each alert: what it means and the first three things to check.

Key terms

- **Monitoring** — continuously measuring a system's health.

- **Structured logging** — logs as machine-readable records such as JSON lines.
- **Latency** — time to answer a request.
- **p95** — the value 95% of measurements are below.
- **Alert rule** — a condition that notifies a person.
- **Runbook** — instructions for responding to an alert.

Remember this: Log every prediction safely, watch service, input, output and quality metrics with percentiles, and set few, specific, actionable alerts.

Name: _____ Date: _____

Activity

Task 1

Add a rule "requests below 200 per day" and simulate a day with only 100 requests.

Task 2

Make day 13 have 30% more rural applicants. Does the approval-rate alert fire?

Task 3

Add a p99 column. How different is it from p95?

Task 4

Write a three-line runbook for the "p95 latency" alert.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why monitor inputs and outputs, not only accuracy?

   A. Accuracy is useless
   B. They are cheaper to print
   C. It is required by Docker
   D. Real outcomes arrive months later, so inputs and outputs are early warnings

2. Why replace the applicant ID with a keyed hash in logs?

   A. Hashes are prettier
   B. To make logs faster
   C. To link logs to outcomes without showing who the person is
   D. To save space

3. What does p95 latency mean?

   A. The slowest request
   B. 95 requests per second
   C. 95% of requests were faster than this value
   D. Average speed

4. Which is a good alert?

   A. Specific, actionable and rare
   B. Sent to nobody
   C. One that fires every hour
   D. Vague and frequent

5. What should never be written to logs?

   A. Model version
   B. Names, phone numbers and national IDs
   C. Status codes
   D. Latency

6. What is a runbook?

   A. A model file
   B. A log format
   C. Instructions for responding to an alert
   D. A training plan for runners

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.